

Rink Reports

Duplicating Rink Reports

How to run your own complete copy of Rink Reports: get the code, create a fresh Supabase backend and load its schema, connect the two, and deploy — with a local-only stopping point for evaluation.

Contents

Duplicating RinkReports — Step-by-Step Setup Guide 3

- Part 0 — Prerequisites (install these once) 3
- Part 1 — Get the application code 3
- Part 2 — Create the Supabase backend 4
- Part 3 — Connect the app to the backend (environment variables) 5
- Part 4 — Generate database types, then run it 5
- Part 5 — Verify the copy is healthy (tests & checks) 6
- Part 6 — Bootstrap the first administrator 6
- Part 7 — Deploy the web app (Vercel) 7
- Part 8 — Email delivery (Resend) — optional 7
- Part 9 — Custom domain & analytics — optional 7
- Part 10 — Confirm the live copy 8
- Quick duplication checklist 8
- Troubleshooting 8

Duplicating RinkReports — Step-by-Step Setup Guide

This guide explains how to stand up a **complete, independent copy** of RinkReports — your own running instance with its own database, its own login, and (optionally) its own public web address. Follow it top to bottom for a fresh production deployment, or stop after Part 4 if you only need a local copy for development or evaluation.

What you are duplicating. RinkReports is a Next.js (App Router) progressive web app backed by a Supabase project (PostgreSQL + Auth). The web app is stateless and hosts on Vercel; all data, accounts, and security rules live in Supabase. So "duplicating the software" really means three things: (1) get the application code, (2) create a fresh Supabase backend and load its database structure, and (3) connect the two with a handful of settings, then deploy. Brand reference for any copy you customize: **#4DFF00** (primary green), **#002244** (navy).

A copy is fully isolated: each instance has its own facilities, employees, and reports — you only ever see data for your own instance and facility; this is automatic.

Part 0 — Prerequisites (install these once)

Tool	Version	Why	Get it
Node.js	20 LTS or newer	Runs the build and tooling	nodejs.org
pnpm	latest	Package manager (this repo's lockfile is <code>pnpm-lock.yaml</code>)	<code>corepack enable pnpm</code>
Git	any recent	Clone the code	git-scm.com
Supabase CLI	latest	Create/link the database, run migrations locally	supabase.com/docs/guides/cli
Docker Desktop	latest	Required by supabase start for the local stack	docker.com
psql (PostgreSQL client)	15+	Run the database security test	bundled with PostgreSQL

Accounts you'll need for a **production** copy (skip for local-only):

- A **Supabase** account (Pro plan recommended for production — log retention and point-in-time recovery).
- A **Vercel** account (to host the web app).
- A **Resend** account (optional) — only if your copy should send invitation/notification emails.
- A **domain name** (optional) — only if you want a custom web address.

Part 1 — Get the application code

```
git clone <your-fork-or-repo-url> rinkreports
cd rinkreports
pnpm install
```

`pnpm install` reads `pnpm-lock.yaml` and installs the exact dependency versions this app was built and tested against (Next.js 16, React 19, the Supabase client libraries, and the test/build tooling).

Tip — make it truly yours. If this is a brand-new copy rather than a contribution back to the original, create your own empty Git repository and push the code there (`git remote set-url origin <your-new-repo>; git push -u origin main`). Your new Supabase project and Vercel project will connect to that repo.

Part 2 — Create the Supabase backend

You have two tracks. Do **Track A** to develop/evaluate on your laptop. Do **Track B** to run a real, always-on copy. For a production copy you'll typically do A first (to verify everything works), then B.

Track A — Local backend (laptop copy)

1. Start Docker Desktop.
2. Boot the local Supabase stack. This **also applies every migration** in `supabase/migrations/` automatically:

```
supabase start
```

It prints local connection details. The committed `supabase/config.toml` pins the local ports: **API 54321, DB 54322, Studio 54323, Inbucket (test email) 54324**. Studio (the database UI) is at `http://127.0.0.1:54323`.

3. The local database now contains the full RinkReports schema — every table, security rule, and helper function — because the migrations ran during `supabase start`.

Track B — Remote backend (production copy)

1. In the Supabase dashboard, **create a new project**. Choose a strong database password and a region near your users.
2. From **Settings** → **API**, copy these three values (you'll need them in Part 3):
 - **Project URL**
 - **anon public key**
 - **service_role secret key** (server-only — never exposed to the browser)
3. Link your local checkout to the new remote project and push the database structure:

```
supabase link --project-ref <your-project-ref>
supabase db push      # applies all migrations in numeric order
```

This creates the entire RinkReports schema on your new cloud database. The migration set is a flat, numerically ordered series of files under `supabase/migrations/` (there are 160+); `supabase db push` runs any that haven't been applied yet.

Important: never hand-edit the cloud database structure. All schema changes go through the migration files so any copy can be rebuilt identically.

Part 3 — Connect the app to the backend (environment variables)

Copy the template and fill it in:

```
cp .env.example .env.local
```

For **local** development, point at the local stack values that supabase start printed. For a **remote/production** copy, use the values from your Supabase project's **Settings → API**.

Variable	Required?	What it is
NEXT_PUBLIC_SUPABASE_URL	Yes	Your Supabase Project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	The Supabase anon public key
SUPABASE_SERVICE_ROLE_KEY	Server features	Service-role secret. Powers the email-invite flow and the notification cron. Leave blank in local dev — the Invite button then shows a friendly error and the cron returns a harmless 503.
NEXT_PUBLIC_SITE_URL	Recommended	Base URL for links inside invitation / password-reset emails. Local: <code>http://localhost:3000</code> . Production: your canonical site URL, no trailing slash.
CRON_SECRET	Production	A long random string (<code>openssl rand -hex 32</code>) that authenticates the scheduled background jobs.
RESEND_API_KEY / RESEND_FROM	Optional	Email delivery (see Part 7). Leave blank to disable — outgoing messages simply queue until configured.
RESEND_ENABLED	Optional	Override the "production only" email gate (<code>true</code> to force-enable, <code>false</code> as a kill switch).
NEXT_PUBLIC_POSTHOG_KEY / _HOST / _ENABLED	Optional	Error/analytics reporting (see Part 8). Leave blank to disable entirely.

Never prefix a secret with `NEXT_PUBLIC_`. Anything with that prefix is shipped to the browser. The service-role key and cron secret must stay server-only.

Part 4 — Generate database types, then run it

1. Regenerate the TypeScript database types so the app's types match your schema. Point `DATABASE_URL` at a fully-migrated database (the local stack is easiest):

```
DATABASE_URL="postgresql://postgres:postgres@127.0.0.1:54322/postgres" pnpm types:write
```

(`pnpm types:check` is the read-only version that CI uses to confirm the types are in sync.)

2. Start the app:

```
pnpm dev          # development server at http://localhost:3000
```

For a production-style local run: `pnpm build` then `pnpm start`.

At this point you have a working copy — but no one can log in yet, because RinkReports has **no public signup**. Create the first account in Part 6.

Part 5 — Verify the copy is healthy (tests & checks)

Run these against your new instance before trusting it:

```
pnpm lint          # code/style checks
pnpm test          # unit tests (vitest) — pure submission/compute logic
pnpm check:cron    # validates the scheduled-job configuration
```

Database security regression test — confirms each facility's data stays isolated (the core guarantee of the app). Run it against a migrated database; it executes in a single transaction and rolls back at the end, so it changes nothing:

```
psql "postgresql://postgres:postgres@127.0.0.1:5432/postgres" \
-v ON_ERROR_STOP=1 -f supabase/tests/rls_isolation.sql
```

Against a **remote** database, point `psql` at that project's connection string instead. Any failure raises and exits non-zero.

Part 6 — Bootstrap the first administrator

There is no public sign-up page, so the very first account is created directly in Supabase. This person becomes the platform owner (super-admin) who can then create facilities and invite everyone else through the app.

1. Create the user. In **Supabase Studio** → **Authentication** → **Users**, click **Add user** and set an email + password. (Locally, Studio is at `http://127.0.0.1:54323`; test emails land in Inbucket at `http://127.0.0.1:54324`.)
2. Promote that user to super-admin. In the **SQL editor**, run:

```
update public.users
set is_super_admin = true
where id = '<the-new-user-uuid>';
```

3. Sign in to the running app with that email/password. You now have the **Super Admin** console and the **Admin Center**.

From here, everything else is done **inside the app** — no more SQL. Follow

ONBOARDING-ADMIN.md to create your first facility, add employees, assign roles, and turn on the modules you need. The detailed configuration for each module lives in its chapter under `modules/` (see the Master Manual).

You can stop here for a local or internal copy. Parts 7–10 are only needed to put your copy on the public internet.

Part 7 — Deploy the web app (Vercel)

1. In Vercel, **Add New → Project** and import your Git repository. Framework preset: **Next.js** (auto-detected).
2. Add the environment variables from Part 3 under **Settings → Environment Variables**:

Variable	Scope
NEXT_PUBLIC_SUPABASE_URL	All environments
NEXT_PUBLIC_SUPABASE_ANON_KEY	All environments
SUPABASE_SERVICE_ROLE_KEY	Production + Preview
CRON_SECRET	Production + Preview
NEXT_PUBLIC_SITE_URL	Per environment (production = your canonical URL)
RESEND_API_KEY / RESEND_FROM	Production + Preview (if using email)

3. **Scheduled background jobs** are committed in `vercel.json` and picked up automatically — no dashboard setup. Your copy runs **five** crons, all authenticated with `CRON_SECRET`:

Schedule (UTC)	Job	Purpose
* / 5 * * * *	/api/cron/drain-notifications	Fan out in-app inbox notifications
2-59 / 5 * * * *	/api/cron/send-communications	Send queued emails via Resend (staggered after drain)
4-59 / 10 * * * *	/api/cron/expire-scheduling	Expire stale scheduling requests
3-59 / 5 * * * *	/api/cron/run-reminders	Send scheduled reminders
17 3 * * *	/api/cron/run-retention-purge	Daily data-retention cleanup, off-peak

4. Deploy (push to main, or trigger from the Vercel UI) and confirm a green build.

Part 8 — Email delivery (Resend) — optional

Only needed if your copy should email staff (login invitations, notifications). Without it, the app works fully; outgoing messages just stay queued and flush automatically once email is configured.

1. In Resend, **verify your sending domain**. The exact DNS records (SPF, DKIM, DMARC) are listed in `.env.example` next to `RESEND_FROM`; verification fails unless all three are present.
2. Generate a Resend API key → `RESEND_API_KEY`.
3. Set `RESEND_FROM` to a sender on the verified domain, e.g. RinkReports `<noreply@send.example.com>`.

Email only sends in the **production** environment by default, so a preview or dev copy that inherits production secrets can never email real staff. Override with `RESEND_ENABLED=true/false` if you need to.

Part 9 — Custom domain & analytics — optional

- **Domain:** add it under Vercel **Settings → Domains**, point your DNS as instructed, and set `NEXT_PUBLIC_SITE_URL` to that canonical address (no trailing slash) so email links resolve correctly. Vercel issues TLS automatically.

- **Analytics/error reporting (PostHog):** set `NEXT_PUBLIC_POSTHOG_KEY` (and `_HOST` only for EU/self-hosted). Leave blank to disable — PostHog is then never loaded, so there's no bundle or network cost. Capture is production-gated and PII-scrubbed.

Part 10 — Confirm the live copy

After the first production deploy, run the smoke tests. The first checks are also automated by the post-deploy-smoke GitHub workflow; the full list is in `../DEPLOY.md` §5.

```
curl -s https://<your-domain>/api/health # 200, {"ok":true,...}
curl -sI https://<your-domain>/login | head -1 # 200
curl -sI https://<your-domain>/sw.js | head -1 # 200 (service worker — enables offline)
curl -sI https://<your-domain>/robots.txt | head -1 # 200, body "Disallow: /"
```

Then sign in as your super-admin and walk the first-facility setup in `ONBOARDING-ADMIN.md`.

Quick duplication checklist

```
[ ] Install Node 20+, pnpm, Git, Supabase CLI, Docker, psql
[ ] git clone ... && pnpm install
[ ] Supabase: supabase start (local) OR create project + supabase link + supabase db push (
remote)
[ ] cp .env.example .env.local → fill SUPABASE URL + anon key (+ service role / cron secret
for prod)
[ ] DATABASE_URL=... pnpm types:write
[ ] pnpm dev (verify it loads)
[ ] pnpm lint && pnpm test && pnpm check:cron
[ ] psql ... -f supabase/tests/rls_isolation.sql (security test passes)
[ ] Create first user in Studio → set is_super_admin = true
[ ] (prod) Import repo to Vercel + set env vars; crons load from vercel.json
[ ] (prod) Configure Resend + DNS; add custom domain; set NEXT_PUBLIC_SITE_URL
[ ] (prod) Run /api/health + smoke tests
[ ] Sign in as super-admin → follow ONBOARDING-ADMIN.md to stand up a facility
```

Troubleshooting

Symptom	Likely cause / fix
supabase start fails	Docker isn't running, or ports 54321–54324 are taken. Start Docker; stop the conflicting service.
App loads but every page bounces to <code>/login</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>ANON_KEY</code> missing or wrong in <code>.env.local</code> .
Can't sign in at all	No account exists yet — RinkReports has no public signup. Do Part 6.
Signed in but <code>/admin</code> shows "Forbidden"	The account isn't an admin. Set <code>is_super_admin = true</code> , or have an admin grant Admin-Center access.
Invite employee button errors	<code>SUPABASE_SERVICE_ROLE_KEY</code> not set (expected in local dev).
<code>pnpm types:check</code> fails in CI	Schema and generated types drifted — rerun <code>pnpm types:write</code> against a fully-migrated DB and commit the result.

Symptom	Likely cause / fix
Emails never arrive	Resend not configured, domain unverified, or you're not in the production environment. See Part 8.
Background jobs not running	CRON_SECRET missing on Vercel, or you're testing on a plan without Cron. Crons are defined in <code>vercel.json</code> .

Companion documents: Master Manual · Admin onboarding · production operations runbook
../DEPLOY.md · local-dev quickstart in the repository README.md.